

Youth Ministry Management App

Database Schema & Table Structure

Youth Ministry Management App - Database Schema

Companion to REQUIREMENTS.md. This is the implementable Postgres/Supabase schema: tables, types, constraints, indexes, and representative Row-Level Security policies. This document is a design artifact for review - **no implementation or migration steps have been run**; nothing here has been executed against a real database yet.

Target: Supabase (Postgres 15+). Conventions used throughout: `uuid` primary keys via `gen_random_uuid()` (pgcrypto/pgcrypto-equivalent, available by default on Supabase), `timestamptz` for all timestamps, `text` in place of `varchar` (idiomatic Postgres), soft-delete via status/archived flags rather than hard deletes except where explicitly noted.

0. Enumerated Types

```
create type app_role as enum ('admin', 'general_coordinator', 'sub_coordinator', 'servant');
create type member_status as enum ('active', 'archived'); create type calendar_event_type as enum
('Trip', 'Outing', 'Group Discussion', 'Speaker Session', 'Event', 'Holiday'); create type
proximity_type as enum ('Local', 'Regional', 'Abroad', 'Unknown'); create type attendee_kind as
enum ('member', 'servant');
```

1. app_settings

Single-row configuration table. (A key/value table is an acceptable alternative if more settings get added later; a single row is simplest for a single-tenant deployment.)

```
create table app_settings ( id boolean primary key default true, -- singleton pattern constraint
app_settings_singleton check (id = true), app_title_long text not null default 'Service Members
Ministry', app_title_short text not null default 'Members Ministry', app_subtitle text not null
default 'Servant Dashboard', logo_url text, -- Supabase Storage path theme_color text not null
default '#1e3a5f', group_label text not null default 'Group', -- "Cohort" for this deployment
member_label text not null default 'Member', -- "Youth" for this deployment group_name_template
text default '{cohort_year} Cohort - Yr {position_label}', current_service_year_label text, --
e.g. "25-26", used in QR labels app_version text not null default '4.0', same_day_cutoff_time
time not null default '21:00', timezone text not null default 'America/New_York', service_weekday
smallint not null default 5 -- 1=Mon .. 7=Sun (5=Friday) check (service_weekday between 1 and 7),
updated_at timestamptz not null default now() ); insert into app_settings (id) values (true);
```

2. groups

Each row is a permanent cohort. `ladder_position` is what the Group Transition process advances; `name` is either auto-regenerated from `group_name_template` (when `cohort_year` is set) or manually managed.

```
create table groups ( id uuid primary key default gen_random_uuid(), cohort_year integer, --
nullable: null = not using cohort-year naming ladder_position smallint not null, -- 0 =
pre-entry, 1..(N-1) = progressing, N = terminal name text not null, is_terminal boolean generated
always as (ladder_position >= 5) stored, is_archived boolean not null default false, -- true once
fully archived out (post-terminal cleanup) display_order integer not null, -- for stable UI
```

```
ordering independent of position ties created_at timestampz not null default now(), unique
(cohort_year) -- one row per cohort year, where cohort_year is used ); create index
idx_groups_ladder_position on groups (ladder_position) where not is_archived;
```

Notes: - The pre-entry group (`ladder_position = 0`) is created and managed only by Admins (enforced via RLS, Section 7). - A "terminal" group (`ladder_position >= 5`) is never advanced further by the transition function - see Section 8. - The UI's aggregate terminal display ("2003 Cohort and earlier - Yr 5+") is **computed at query time**, not stored: `sql select min(cohort_year) as oldest_open_cohort from groups where is_terminal and not is_archived;`

3. members

```
create table members ( id uuid primary key default gen_random_uuid(), group_id uuid not null
references groups(id), photo_path text, -- Supabase Storage object path full_name text not null,
phone text, email text, university_id uuid references universities(id), program_of_study text,
date_of_birth date, father_of_confession text, home_address text, is_visitor boolean not null
default false, gender text check (gender in ('Male', 'Female')), registration_comments text, --
read-only after creation at the app layer assigned_servant_id uuid references profiles(id),
servant_comments text, is_new_assignment boolean not null default false, status member_status not
null default 'active', legacy_source_ref text, -- migration tracking, see DATA MIGRATION note
(Section 11) created_at timestampz not null default now(), updated_at timestampz not null
default now() ); create index idx_members_group on members (group_id) where status = 'active';
create index idx_members_assigned_servant on members (assigned_servant_id); create index
idx_members_legacy_ref on members (legacy_source_ref); create unique index uq_members_legacy_ref
on members (legacy_source_ref) where legacy_source_ref is not null;
```

`universities` must be declared before `members` in actual execution order, or the FK added after both tables exist - shown here in logical reading order.

4. universities

```
create table universities ( id uuid primary key default gen_random_uuid(), name text not null
unique, proximity proximity_type not null default 'Unknown' );
```

5. profiles (extends Supabase `auth.users` - Admins, Coordinators, Servants; never Members)

Supabase Auth owns `auth.users` (id, email, auth metadata). `profiles` holds the app-specific fields, one row per authenticated user, created via a trigger on signup.

```
create table profiles ( id uuid primary key references auth.users(id) on delete cascade,
full_name text not null, phone text, email text, -- denormalized copy of auth.users.email for
convenience father_of_confession text, gender text check (gender in ('Male', 'Female')),
photo_path text, created_at timestampz not null default now() );
```

6. user_roles

The heart of the permission model. A user can hold multiple rows, including multiple rows of the *same* role type scoped to different groups (this is how cross-group "exception access" is granted - see REQUIREMENTS.md Section 4.2).

```
create table user_roles ( id uuid primary key default gen_random_uuid(), user_id uuid not null
references profiles(id) on delete cascade, role app_role not null, group_id uuid references
groups(id), created_at timestamptz not null default now(), constraint role_group_scope_check
check ( (role in ('admin', 'general_coordinator') and group_id is null) or (role in
('sub_coordinator', 'servant') and group_id is not null) ), unique (user_id, role, group_id) --
prevents an identical duplicate grant; different groups are fine ); create index
idx_user_roles_user on user_roles (user_id); create index idx_user_roles_group on user_roles
(group_id);
```

Helper functions (used throughout RLS policies, Section 7):

```
create or replace function is_admin_or_general_coordinator(uid uuid default auth.uid()) returns
boolean language sql stable security definer as $$ select exists ( select 1 from user_roles where
user_id = uid and role in ('admin', 'general_coordinator') ); $$; create or replace function
is_admin(uid uuid default auth.uid()) returns boolean language sql stable security definer as $$
select exists (select 1 from user_roles where user_id = uid and role = 'admin'); $$; create or
replace function has_group_access(gid uuid, uid uuid default auth.uid()) returns boolean language
sql stable security definer as $$ select is_admin_or_general_coordinator(uid) or exists ( select
1 from user_roles where user_id = uid and group_id = gid ); $$; create or replace function
can_manage_servants(uid uuid default auth.uid()) returns boolean language sql stable security
definer as $$ -- Reassigning a servant's group, or removing a servant: General Coordinator /
Admin only select is_admin_or_general_coordinator(uid); $$;
```

7. Row-Level Security (representative policies)

Enable RLS on every table holding ministry data; deny by default; grant explicitly.

```
alter table members enable row level security; create policy members_select on members for select
using (has_group_access(group_id)); create policy members_update on members for update using
(has_group_access(group_id)); create policy members_insert on members for insert with check
(has_group_access(group_id)); -- Permanent delete: Admins only, any group (correction tool,
REQUIREMENTS.md Section 3.3.1) create policy members_delete on members for delete using
(is_admin());
```

```
alter table groups enable row level security; -- Position-0 (pre-entry) groups are visible only
to Admins; all other positions -- follow the normal group-access rule. create policy
groups_select on groups for select using ( (ladder_position = 0 and is_admin()) or
(ladder_position > 0 and has_group_access(id)) ); create policy groups_admin_write on groups for
all using (is_admin()) with check (is_admin());
```

```
alter table user_roles enable row level security; create policy user_roles_select on user_roles
for select using (is_admin_or_general_coordinator() or user_id = auth.uid()); create policy
user_roles_admin_write on user_roles for all using (is_admin()) with check (is_admin());
```

```
alter table attendance_records enable row level security; create policy attendance_select on
attendance_records for select using ( (attendee_type = 'member' and has_group_access( (select
```

```
group_id from members where members.id = member_id))) or (attendee_type = 'servant' and
is_admin_or_general_coordinator()) ); create policy attendance_write on attendance_records for
insert with check ( (attendee_type = 'member' and has_group_access( (select group_id from members
where members.id = member_id)))) );
```

(Every remaining table - *outreach_entries*, *service_calendar_events*, *actions_needed_config*, *audit_log*, *audit_config*, *qr_codes*, *verses*, *app_settings*, *universities* - follows the same pattern: *has_group_access()* for group-scoped tables, *is_admin()* for admin-only tables, and *is_admin_or_general_coordinator()* for the servant-management actions called out in *REQUIREMENTS.md* Section 4.1/Section 6.13. The implementing agent should write the exact policy per table following these three helper functions rather than inventing new patterns - this keeps the whole permission model auditable in one place.)

Servant self-reassignment restriction (*REQUIREMENTS.md* Section 6.13 - Sub-Coordinators cannot reassign or remove a servant):

```
alter table user_roles enable row level security; -- already enabled above -- Only General
Coordinators/Admins may write a user_roles row of type 'servant' -- that changes an existing
servant's group scope, or delete a servant's role row. create policy
user_roles_servant_management on user_roles for update using (role = 'servant' and
can_manage_servants()) with check (role = 'servant' and can_manage_servants());
```

8. attendance_records

```
create table attendance_records ( id uuid primary key default gen_random_uuid(), attendee_type
attendee_kind not null, member_id uuid references members(id), servant_id uuid references
profiles(id), service_date date not null, is_visitor_at_time boolean not null default false, --
snapshot, in case visitor status changes later created_at timestamptz not null default now(),
constraint attendance_exactly_one_attendee check ( (attendee_type = 'member' and member_id is not
null and servant_id is null) or (attendee_type = 'servant' and servant_id is not null and
member_id is null) ), unique (member_id, service_date), unique (servant_id, service_date) );
create index idx_attendance_member_date on attendance_records (member_id, service_date); create
index idx_attendance_servant_date on attendance_records (servant_id, service_date); create index
idx_attendance_service_date on attendance_records (service_date);
```

A row's mere existence = present on that date. Absence is inferred (no row for a date the group was tracking). "Tracked dates" for a group = any *service_date* with at least one attendance row for a member of that group.

9. outreach_entries

```
create table outreach_entries ( id uuid primary key default gen_random_uuid(), member_id uuid not
null references members(id), servant_id uuid not null references profiles(id), occurred_at
timestamptz not null default now(), type text, notes text, follow_up_due date,
follow_up_dismissed_at timestamptz, created_at timestamptz not null default now() ); create index
idx_outreach_member on outreach_entries (member_id); create index idx_outreach_servant on
outreach_entries (servant_id); create index idx_outreach_follow_up on outreach_entries
(follow_up_due) where follow_up_dismissed_at is null;
```

RLS: only the creating servant (*servant_id* = *auth.uid()*) may update/delete their own rows; anyone with group access to the member may *select/insert*.

10. service_calendar_events

```
create table service_calendar_events ( id uuid primary key default gen_random_uuid(), title text not null, description text, event_type calendar_event_type not null, start_date date not null, end_date date not null, all_day boolean not null default true, start_time time, end_time time, location text, attachment_url text, created_by uuid not null references profiles(id), created_at timestampz not null default now(), check (end_date >= start_date) ); create index idx_calendar_events_dates on service_calendar_events (start_date, end_date);
```

RLS: any authenticated Servant (or higher) may insert/update/delete - matches the confirmed "open to all servants" decision.

11. actions_needed_config

```
create table actions_needed_config ( proximity proximity_type primary key, min_presence_count integer not null default 0, min_absence_weeks integer not null default 3, min_outreach_weeks integer not null default 4 ); insert into actions_needed_config (proximity, min_presence_count, min_absence_weeks, min_outreach_weeks) values ('Local', 0, 3, 4), ('Regional', 0, 3, 4), ('Abroad', 0, 6, 4), ('Unknown', 0, 3, 4);
```

RLS: readable by anyone with app access (feeds the live Dashboard help modal, REQUIREMENTS.md Section 6.9); writable by Admins only.

12. audit_log and audit_config

```
create type audit_action_type as enum ( 'APP_ACCESS', 'GROUP_SELECTED', 'MEMBER_EDITED', 'SERVANT_ASSIGNED', 'OUTREACH_ADDED', 'OUTREACH_UPDATED', 'OUTREACH_DELETED', 'MEMBER_PHOTO_UPLOADED', 'SERVANT_PROFILES_VIEWED', 'SERVANT_ATTENDANCE_VIEWED', 'SERVANT_EDITED', 'SERVANT_GROUP_UPDATED', 'SERVANT_PHOTO_UPLOADED', 'SERVANT_DELETED', 'ADMIN_ACCESS_MAINTENANCE', 'ADMIN_UNIVERSITIES_MAINTENANCE', 'ATTENDANCE_ADDED', 'ATTENDANCE_REMOVED', 'CALENDAR_EVENT_CREATED', 'CALENDAR_EVENT_UPDATED', 'CALENDAR_EVENT_DELETED', 'MEMBER_ARCHIVED', 'MEMBER_DELETED', 'GROUP_TRANSITION_RUN' ); create table audit_config ( action_type audit_action_type primary key, enabled boolean not null default true, description text ); create table audit_log ( id bigint generated always as identity primary key, occurred_at timestampz not null default now(), user_id uuid references profiles(id), action_type audit_action_type not null, group_id uuid references groups(id), details jsonb ); create index idx_audit_log_time on audit_log (occurred_at desc); create index idx_audit_log_user on audit_log (user_id); create index idx_audit_log_action on audit_log (action_type);
```

RLS: audit_log/audit_config readable and writable by Admins only. Application code should check audit_config.enabled before writing a row of that type (matches current app's per-action toggle behavior).

13. qr_codes

```
create type qr_flow_type as enum ('check_in_and_intake', 'intake_only'); create table qr_codes (
id uuid primary key default gen_random_uuid(), group_id uuid references groups(id), -- null = the
"Servants" QR label text not null, image_path text not null, -- Supabase Storage object path
check_in_token text not null unique, -- opaque token forming the public check-in URL flow_type
qr_flow_type not null default 'check_in_and_intake', printed_at timestampz, -- null/stale
relative to group.updated_at = needs reprint updated_at timestampz not null default now() );
```

`flow_type` distinguishes the position-0 (pre-entry) group's QR - which must only ever open the intake/registration form (`intake_only`, no attendance option, since that group isn't tracked for attendance) - from every other group's QR, which supports both the existing-member check-in flow and the new-member intake flow (`check_in_and_intake`). A newly-created group defaults to `check_in_and_intake`; a group at `ladder_position = 0` should be created with `intake_only`.

A QR code "needs reprinting" whenever `printed_at` is null or `printed_at < groups.updated_at` for its `group_id` (or, for the Servants code, whenever its own label changes). The Group Transition process (Section 8 below) should touch `groups.updated_at` whenever a name regenerates, making this comparison trivial.

(Add `updated_at timestampz not null default now()` to `groups`, maintained by a standard before update trigger - omitted above for brevity, required in the actual migration.)

14. verses

```
create table verses ( id uuid primary key default gen_random_uuid(), text text not null,
reference text, -- e.g. "John 3:16" is_active boolean not null default true );
```

Application logic: select * from verses where `is_active` order by `random()` limit 1 whenever "Load [Member] Data" is clicked.

15. The Group Transition function

Sketch of the atomic transition procedure (REQUIREMENTS.md Section 5). All steps run inside one transaction - a PL/pgSQL function is the natural way to guarantee this in Postgres, since the entire function body is atomic by default (an unhandled exception anywhere inside it rolls back everything it did).

```
create or replace function run_group_transition(new_pre_entry_cohort_year integer) returns void
language plpgsql security definer as $$ begin -- Guard: admin only (defense in depth; RLS/API
layer should also enforce this) if not is_admin() then raise exception 'Only Admins may run a
Group Transition'; end if; -- Advance every non-terminal group one ladder position. update groups
set ladder_position = ladder_position + 1 where ladder_position < 5 and not is_archived; --
Regenerate names for every group that has a cohort_year (template-driven naming). update groups
set name = replace( replace( (select group_name_template from app_settings), '{cohort_year}',
cohort_year::text ), '{position_label}', case when ladder_position >= 5 then '5+' else
ladder_position::text end ), updated_at = now() where cohort_year is not null and not
is_archived; -- Create the new pre-entry (position 0) cohort for the upcoming intake year. insert
into groups (cohort_year, ladder_position, name, display_order) values (
new_pre_entry_cohort_year, 0, replace( replace((select group_name_template from app_settings),
'{cohort_year}', new_pre_entry_cohort_year::text), '{position_label}', '0' ), (select
coalesce(max(display_order), 0) + 1 from groups )); -- Sub-Coordination and Servants scoped to a
group automatically follow it, since -- user_roles.group_id references the same group row whose
position just advanced - -- no separate update needed here; this is the mechanism, not a side
```

```
effect to compute. insert into audit_log (user_id, action_type, details) values (auth.uid(),
'GROUP_TRANSITION_RUN', jsonb_build_object('new_pre_entry_cohort_year',
new_pre_entry_cohort_year)); end; $$;
```

If any statement in this function raises an exception, Postgres automatically rolls back every change the function made - satisfying the "no half-baked transition" requirement without any additional application-level rollback logic.

The **optional post-transition servant review** and **QR-reprint prompt** (REQUIREMENTS.md Section 5) are UI-layer flows that run *after* this function successfully commits - they read the now-updated `groups`/`user_roles`/`qr_codes` tables to build their checklists, but are not part of the atomic transaction itself (they're advisory follow-up steps, not data-integrity-critical ones).

16. Entity Relationship Summary

```
groups (1) -----< members (many) groups (1) -----< user_roles (many, where role in
sub_coordinator/servant) groups (1) -----< qr_codes (many, one row typically; null group_id =
Servants QR) profiles (1) -----< user_roles (many) profiles (1) -----<
members.assigned_servant_id (many) profiles (1) -----< outreach_entries.servant_id (many)
profiles (1) -----< attendance_records.servant_id (many) profiles (1) -----<
service_calendar_events.created_by (many) members (1) -----< attendance_records (many) members
(1) -----< outreach_entries (many) universities (1) ---< members (many) actions_needed_config,
audit_config, audit_log, verses, app_settings - standalone (audit_log references profiles/groups
but nothing references it)
```

17. `legacy_source_ref` - one-way migration tracking

REQUIREMENTS.md Section 10.1 confirms the current Google Sheets app remains the **sole source of truth** throughout the testing period - the sync is one-way (Sheets -> this database). Each refresh makes the *operational* tables an exact mirror of current Sheets content; *configuration* tables are seeded once and then excluded from the ongoing sweep (revised in v5 to broaden this exclusion beyond just role assignments). No separate tracking table is needed - just a `legacy_source_ref` column on every operational table.

Operational tables - swept on every refresh. Add `legacy_source_ref` to each:

```
alter table <table_name> add column legacy_source_ref text; create unique index
uq_<table_name>_legacy_ref on <table_name> (legacy_source_ref) where legacy_source_ref is not
null;
```

Applies to: `members` (already declared directly in its `create table`, Section 3 above - shown here as the general pattern), `attendance_records`, `outreach_entries`, `service_calendar_events`, and `profiles` (contact-info fields only).

Refresh algorithm (run by an external migration tool via the Sheets API, not by the app at runtime), per operational table: 1. Read the current contents of the corresponding sheet/tab. 2. `insert ... on conflict (legacy_source_ref) do update` - upsert every current Sheets row, keyed by `legacy_source_ref`. New Sheets rows get inserted; existing ones get updated to match the Sheets values exactly. 3. `delete from <table> where legacy_source_ref is not null and legacy_source_ref not in (<refs seen in this refresh>)` - removes rows whose Sheets source disappeared. 4. `delete from <table> where legacy_source_ref is null` - removes

any row created directly in the new app (no Sheets origin at all), i.e. disposable test data from exploring the app between refreshes.

Configuration tables - excluded from the ongoing sweep (broadened in v5). `universities`, `verses`, `actions_needed_config`, and `audit_config` still get a `legacy_source_ref` column and are populated once from their current Sheets values during the **initial** migration only - never revisited by a subsequent refresh. Each has its own admin maintenance screen in the new app (Section 6.14, Section 6.9, Section 6.1), and edits made there are meant to stick, not get overwritten on the next "refresh the data" request. Trade-off: a brand-new university or verse added on the Sheets side after initial migration won't auto-appear via refresh - it needs to be added directly through the new app's own maintenance screen instead.

`user_roles` is likewise **not** synced this way - it's a new-app-native concept (Section 4, Section 6), set up once via Access Maintenance and expected to diverge permanently from the old Permissions sheet, not mirrored from it.

`groups`, `app_settings`, `qr_codes`, and `audit_log` have no Sheets equivalent at all and are outside this refresh process entirely - they're set up once and evolve only through the app's own tools.

No tables, functions, or policies in this document have been created in any real database. This is a design specification for review alongside REQUIREMENTS.md.